

1. TrackServ: Citizen Application

Citizen Application is the public-facing portal.

Its goals are to:

- Let residents register and log in.
- Submit municipal service reports.
- Add descriptions, locations, coordinates, and photos.
- Track report status.
- Vote on reports.
- Participate in community discussions.
- View their profile and submitted reports.

The main data flow is:

1. A citizen submits a report to the reports table.
2. The selected category is resolved through categories.
3. Photos are uploaded to the Supabase images bucket.
4. Photo records are stored in report_images.
5. Citizens can later see status and activity from the same database.

The reporting implementation is visible in ReportIssues.jsx:362-430.

This is the strongest product-facing application. It expresses the core purpose of TrackServ: giving citizens a structured channel to report problems and monitor public services.

One limitation is that the dashboard uses internal state for pages such as reports, profile, community, and issue submission instead of separate URLs. That makes browser history, deep links, and refresh behavior less reliable.

2. admin-dashboard: Municipal Operations Portal

Municipal Operations Portal is an administrative control center.

Its goals are to:

- Review incoming citizen reports.
- Filter and prioritize issues.
- Change report statuses.
- Assign reports to staff.
- Schedule visits or events.
- Analyze report trends.
- Manage locations and operational information.

The connection to TrackServ is clear in `ReportsPage.jsx:380-444`. It reads the same reports, categories, profiles, and `report_images` data created by the citizen application.

Assignments are written back to `reports.assigned_to`, while scheduled work is recorded in events, as shown in `AssignmentsPage.jsx:975-1045`.

The admin portal also subscribes to Supabase realtime changes, so report updates can appear without a complete manual refresh.

The main weakness is the role check in `AdminLogin.jsx:64-82`. It rejects citizens, but accepts every other role. The intended policy appears to be specifically admin and moderator, so it should use an explicit allowlist.

3. TrackServers: Staff Application

It is a Vite React application for field staff.

App.jsx:1-18 defines the staff login, dashboard, and profile routes.

Its goals are to:

- Show staff members the reports assigned to them.
- Display issue locations on a map.
- Calculate routes from the staff member's location to the issue.
- Let staff update work status.
- Capture resolution notes and completion evidence.
- Maintain a history of completed work.

The staff dashboard only loads reports assigned to the current staff profile in Dashboard.jsx:270-327.

This creates the operational handoff:

1. A citizen creates a report.
2. An administrator reviews it.
3. The administrator assigns it to a staff user.
4. The staff app finds it through assigned_to.
5. The staff member updates its progress.
6. The citizen and administrator can see the updated status.

That is the clearest evidence that the three projects are intended to form one municipal service workflow.

How They Are Connected

All three applications use the same Supabase project and publishable key:

- TrackServ supabaseClient.js
- admin-dashboard supabaseClient.js
- TrackServers supabaseClient.js

The principal shared entities appear to be:

- profiles
- categories
- reports
- report_images
- groups
- events
- issue_resolutions
- Supabase Storage images bucket

Roles form the application boundary:

- citizen uses TrackServ.
- admin and moderator use the admin dashboard.
- staff uses the TrackStaff application.

The frontends do not communicate directly with one another. Supabase is effectively the integration layer.

Deployment Relationship

The applications are also deployed separately:

- TrackServ: TrackServ
- Admin portal: admin-dashboard
- Staff portal: /TrackStaff/

TrackServ and the admin portal use Create React App. TrackStaff uses Vite with a configured base path in vite.config.js:5-9.

Each project has its own GitHub Pages deployment workflow. This is reasonable if they are separate repositories. If they share one repository and the same gh-pages branch, the workflows could overwrite each other because each publishes to the branch root.

The routing strategies differ:

- TrackServ uses BrowserRouter.
- Admin uses HashRouter.
- TrackStaff uses BrowserRouter with a basename.

That inconsistency makes deployment and navigation harder to maintain. Hash routing avoids GitHub Pages refresh problems, while the other two applications require rewrite handling through 404.html or equivalent configuration.

Strengths

- The overall product workflow is coherent.
- The three applications have clearly differentiated users.
- The shared database model is a sensible integration point.
- Supabase realtime is used for administrative report updates.
- Staff assignments are linked through stable profile IDs.
- The system includes useful operational concepts such as severity, assignment, scheduling, routing, evidence, and resolution history.
- The citizen experience includes more than basic reporting: voting and community features support civic engagement.

Main Risks

Route protection is incomplete

The login screens check roles, but the dashboard routes themselves do not appear to enforce authentication. A user can navigate directly to routes such as /home, /dashboard, or /TrackStaff/dashboard.

Frontend checks are not sufficient by themselves. Supabase Row Level Security must prevent:

- Citizens from changing report status.
- Staff from editing unassigned reports.
- Staff from viewing other staff members' assignments.
- Unauthorized users from reading profiles or administrative data.
- Unauthorized uploads or storage access.

Shared browser sessions

If these sites are hosted under the same GitHub Pages origin, Supabase browser sessions may be shared because storage is origin-based rather than path-based. Signing in to one application can therefore affect the others.

The database policies must enforce roles and ownership independently of the frontend.

Reused components

The Supabase client, sidebar, profile logic, role labels, status labels, category metadata, and authentication behavior are repeated across applications.

There are also signs of leftover code:

- A stale top-level App.js appears separate from the active src/App.js.
- AdminLocations.jsx appears to be an unused copy of the admin locations page.
- The admin and staff READMEs still describe template projects rather than the actual applications.

This duplication will cause the portals to drift over time.